

Speeding Up Time to Market *with* Automated Web Application Testing

A well-developed Web app ensures a great user experience. Web application testing is itself dependent on several time tested procedures such as continuous testing, which is a best practice for automating the application's life cycle management.



Continuous testing or CT is one of the most significant DevOps practices for the end-to-end automation of application life cycle management. It is essential to integrate automated tests into application life cycle management early, to test quickly and in a timely manner, and to repeat the test execution process efficiently. However, test automation is the elephant in the room for many organisations. Let us try to understand the different types of testing, with some related diagrams for simplicity.

The main objective of Web application testing is to give end users an experience that is devoid of defects or issues. The testing process also verifies whether the application has been created according to the requirements specified by the customer and whether or not the end user will be happy after using the application. Testing not only ensures better applications but also provides insights into the quality level or efficiency of the development process.

There are many different types of testing and each of these can be sub-categorised as well. However, in this article, we will restrict ourselves to understanding the three main types of Web application testing.

- Functional testing
- Load testing
- Security testing

Functional testing

This helps to verify the functional requirements of an application. Usually, this starts off as a manual effort to understand the requirements and scope of the app, and subsequently, is automated to match with the Agile development and DevOps practices. Functional testing focuses on results.

Shown below is the basic sequence of functional testing:

- We first need to understand the requirements/features of an application.
- We need to identify inputs for our function test execution.
- We need to get the expected results with the test inputs we have selected. We then have to execute a functional test case and save the outcome.
- We need to compare selected test input values with the outcome of the functional test.

Let's now understand functional testing using Selenium.